

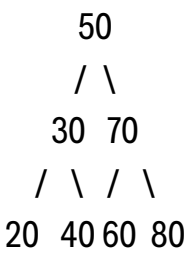
FICHA TÉCNICA ÁRBOL BINARIO DE BÚSQUEDA

NOMBRE: ÁRBOL BINARIO DE BÚSQUEDA

DEFINICIÓN

Estructura de datos jerárquica donde cada nodo tiene como máximo dos hijos. Los valores menores se almacenan a la izquierda y los mayores a la derecha.

VISUALIZACIÓN CONCEPTUAL



CARACTERÍSTICAS MECÁNICAS

- Mantiene los elementos ordenados automáticamente.
- Cada nodo posee:
 - Subárbol izquierdo con valores menores.
 - Subárbol derecho con valores mayores.
- Si el árbol se desbalancea, su rendimiento disminuye considerablemente.

VENTAJAS Y LIMITACIONES

Ventajas

- Búsquedas rápidas cuando el árbol está balanceado.
- Permite recorridos ordenados de datos.

Limitaciones

- Puede degradarse a una lista enlazada si los datos se insertan ordenados.

Caso de Uso en la Industria

Utilizado en sistemas de indexación y búsqueda en memoria, además de compiladores y estructuras de símbolos.

COMPLEJIDAD ALGORÍTMICA

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(n)$
Inserción	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$

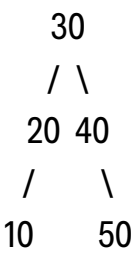
FICHA TÉCNICA ÁRBOL AVL

NOMBRE: ÁRBOL AVL

DEFINICIÓN

Árbol binario de búsqueda auto-balanceado donde la diferencia de altura entre subárboles no puede ser mayor a 1.

VISUALIZACIÓN CONCEPTUAL



CARACTERÍSTICAS MECÁNICAS

- Mantiene balance automático mediante rotaciones:
 - Rotación izquierda.
 - Rotación derecha.
 - Rotaciones dobles.
- Garantiza altura mínima del árbol.

VENTAJAS Y LIMITACIONES

Ventajas

- Excelente rendimiento en búsquedas.
- Siempre mantiene eficiencia logarítmica.

Limitaciones

- Inserciones y eliminaciones más complejas por el rebalanceo.

Caso de Uso en la Industria

Utilizado en bases de datos en memoria, sistemas de archivos y aplicaciones donde las búsquedas son muy frecuentes.

COMPLEJIDAD ALGORÍTMICA

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

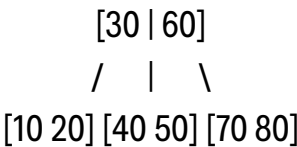
FICHA TÉCNICA ÁRBOL B / B+

NOMBRE: ÁRBOL B Y ÁRBOL B+

DEFINICIÓN

Estructuras balanceadas diseñadas para almacenamiento en disco y bases de datos. Permiten múltiples hijos por nodo.

VISUALIZACIÓN CONCEPTUAL



CARACTERÍSTICAS MECÁNICAS

- Cada nodo puede almacenar múltiples claves.
- Reduce accesos al disco gracias a su alto factor de ramificación.
- El Árbol B+ almacena los datos reales en las hojas.

VENTAJAS Y LIMITACIONES

Ventajas

- Muy eficiente para almacenamiento masivo.
- Reduce operaciones de lectura y escritura en disco.

Limitaciones

- Implementación más compleja que otros árboles.

Caso de Uso en la Industria

Utilizado en motores de bases de datos como PostgreSQL, MySQL y sistemas de archivos modernos.

COMPLEJIDAD ALGORÍTMICA

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

FICHA TÉCNICA TRIE (ÁRBOL DE PREFIJOS)

NOMBRE: TRIE O ÁRBOL DE PREFIJOS

DEFINICIÓN

Estructura especializada para almacenar cadenas de texto, donde cada nodo representa un carácter.

VISUALIZACIÓN CONCEPTUAL

```
( )  
 / \  
c  d  
 |  |  
a  o  
 |  |  
t  g
```

CARACTERÍSTICAS MECÁNICAS

- Organiza palabras carácter por carácter.
- Comparte prefijos comunes para ahorrar tiempo de búsqueda.
- Ideal para búsquedas predictivas.

VENTAJAS Y LIMITACIONES

Ventajas

- Muy rápido para autocompletado y diccionarios.
- Eficiente en búsquedas por prefijo.

Limitaciones

- Consume mucha memoria si hay pocas coincidencias de prefijos.

Caso de Uso en la Industria

Utilizado en motores de búsqueda, correctores ortográficos y sistemas de autocompletado como los de Google.

COMPLEJIDAD ALGORÍTMICA

Operación	Complejidad	
Búsqueda	$O(m)$	
Inserción	$O(m)$	m = longitud de la palabra
Eliminación	$O(m)$	

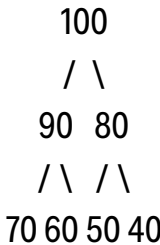
FICHA TÉCNICA HEAP (MONTÍCULO MIN/MAX)

NOMBRE: HEAP O MONTÍCULO

DEFINICIÓN

Estructura de árbol binario completo donde el nodo padre siempre es mayor (Max Heap) o menor (Min Heap) que sus hijos.

VISUALIZACIÓN CONCEPTUAL



CARACTERÍSTICAS MECÁNICAS

- Se implementa comúnmente mediante arreglos.
- Mantiene prioridad automáticamente.
- El elemento mayor o menor siempre está en la raíz.

VENTAJAS Y LIMITACIONES

Ventajas

- Excelente para colas de prioridad.
- Inserciones y eliminaciones rápidas.

Limitaciones

- No es eficiente para búsquedas generales.

Caso de Uso en la Industria

Utilizado en algoritmos de planificación de procesos, sistemas operativos y algoritmos como Dijkstra y HeapSort.

COMPLEJIDAD ALGORÍTMICA

Operación	Complejidad
Búsqueda	O(n)
Inserción	O(log n)
Eliminación	O(log n)